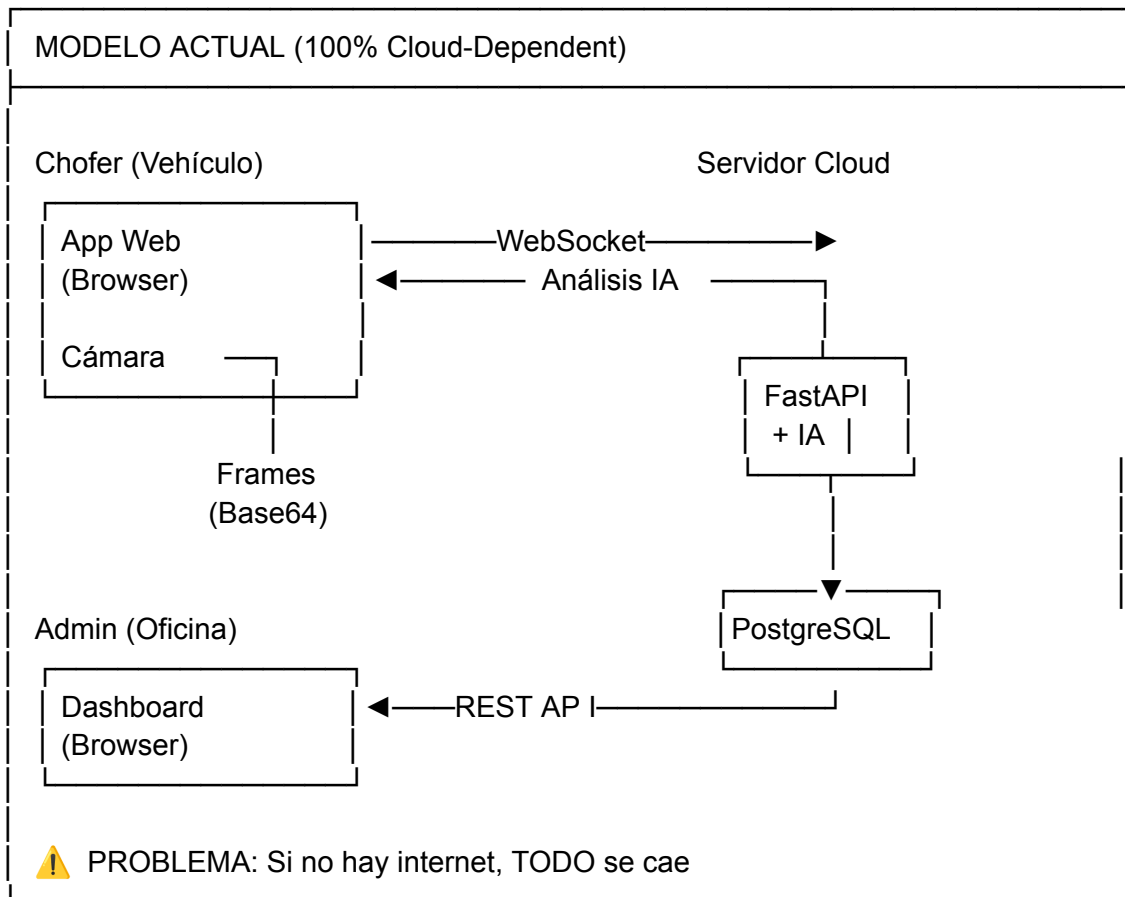


ANÁLISIS DEL MODELO DE NEGOCIO ACTUAL

Lo que tienes ahora:



IDENTIFICACIÓN DE PROBLEMAS CRÍTICOS

PROBLEMA #1: Dependencia Total de Internet 🛑

Escenario Real:

- Chofer entra a zona sin cobertura
- WebSocket se desconecta
- App web deja de procesar frames
- ⚠ NO HAY ALERTA de somnolencia
- 💀 RIESGO DE ACCIDENTE

Severidad: 🔴 **CRÍTICA** - Puede causar accidentes fatales

PROBLEMA #2: Procesamiento en la Nube es Inviabile 🚫

Cálculo de Latencia Real:

Frame capture:	~33ms (30 FPS)
Base64 encode:	~10ms
Upload (4G): 640x480	~50-200ms
IA processing:	~100-300ms
Download response:	~30-100ms
Decode + render:	~20ms
TOTAL LATENCY:	~243-663ms
⚠ Delay de detección: 0.25-0.6 segundos	
A 100 km/h = 27.7 m/s	
En 0.5s avanza: 13.8 metros SIN alerta	

Severidad: 🚫 **CRÍTICA** - Delay inaceptable para seguridad vial

PROBLEMA #3: Costos de Ancho de Banda Prohibitivos 💰

Cálculo de Datos:

- Frame: 640x480 JPEG 80% ≈ 50-80 KB
- FPS: 30 frames/segundo
- Datos/segundo: 1.5 - 2.4 MB/s
- Datos/hora: 5.4 - 8.6 GB/hora
- Viaje 8 horas: 43 - 69 GB
- Plan datos móviles Bolivia: ~20 GB/mes

⚠ UN SOLO VIAJE consume 2-3 MESES de datos

Severidad: 🟡 **ALTA** - Inviabile económicamente

PROBLEMA #4: Monitoreo Centralizado Requiere Infraestructura Compleja 🏗

Para 50 choferes simultáneos:

- Ancho de banda servidor: 75-120 MB/s
- Procesamiento GPU: 50 x (100-300ms) = Necesitas cluster
- Costo AWS/Azure: ~\$2,000-5,000/mes mínimo
- Escalabilidad: Difícil y costosa

SOLUCIÓN ARQUITECTÓNICA RECOMENDADA

ARQUITECTURA HÍBRIDA (Edge + Cloud)

IMPLEMENTACIÓN DETALLADA

OPCIÓN: Aplicación Android Nativa

Si el vehículo tiene Android Auto o tablet:

Hardware:

- └ Tablet Android ($\geq$ Android 10)
- └ Cámara frontal del tablet
- └ Soporte para tablero

Software:

- └ App nativa Android (Kotlin/Flutter)
- └ MediaPipe Android SDK
- └ Base de datos local: Room/SQLite
- └ Sync en background

PROTOCOLO DE SINCRONIZACIÓN CLOUD

Datos que se envían al Cloud (SOLO metadatos)

```
{
  "id_viaje": 123,
  "id_chofer": 45,
  "timestamp": "2025-01-15T14:32:10Z",
  "gps": {
    "lat": -16.5000,
    "lon": -68.1500,
    "velocidad_kmh": 85
  },
  "metrics": {
    "parpadeos_minuto": 18,
    "microsueños_total": 2,
    "bostezos_ultima_hora": 4,
    "inclinaciones_cabeza": 3,
    "frotamiento_ojos": 1
  },
  "alertas": [
    {
      "tipo": "microsueño",
      "timestamp": "2025-01-15T14:30:45Z",
```

```

        "duracion_segundos": 3.2
    }
],
"estado_viaje": "en_curso"
}

```

Tamaño: ~1-2 KB por reporte

Frecuencia: Cada 1-5 minutos

Consumo: ~0.5-1 MB/hora (vs 5-8 GB/hora del modelo actual)

MANEJO DE CONECTIVIDAD

Estados del Sistema:

```
class EstadoConexion:
```

```
    ONLINE = "online"      # WiFi/4G disponible
```

```
    OFFLINE = "offline"    # Sin conexión
```

```
    SYNC_PENDING = "pending" # Datos esperando sync
```

```
# Lógica del Edge Device
```

```
while True:
```

```
    # 1. SIEMPRE procesar localmente
```

```
    frame = capturar_frame()
```

```
    prediccion = modelo_ia.detectar(frame)
```

```
    # 2. Alertar INMEDIATAMENTE si necesario
```

```
    if prediccion.es_critica():
```

```
        reproducir_alarma()
```

```
        guardar_en_buffer_local(prediccion)
```

```
    # 3. Intentar sync (no bloqueante)
```

```
    if hay_conexion() and buffer_tiene_datos():
```

```
        try:
```

```
            enviar_a_cloud_async(buffer.obtener_pendientes())
```

```
            buffer.marcar_como_sincronizado()
```

```
        except ConnectionError:
```

```
            # Silenciosamente fallar, reintentará después
```

```
            pass
```

```
    ...
```

```
---
```

🤖 DASHBOARD ADMIN - FUNCIONALIDAD REALISTA

Vista en Tiempo Real (con delays aceptables)

...

MONITOREO DE FLOTA - VISTA ACTUAL

 Mapa (Google Maps / Leaflet)

 Chofer A (Última actualización:
14:32 - hace 2 min)
Estado:  Alerta Moderada

 Chofer B (Última actualización:
14:33 - hace 1 min)
Estado:  Normal

 Chofer C ( Sin señal
desde 14:20 - hace 14 min)
Última posición conocida

 ALERTAS RECIENTES

 Chofer A - Microsueño (3.2s)
14:30:45 - Ruta La Paz-Oruro

 Chofer D - 4 bostezos (última hr)
14:25:10 - Ruta Cochabamba-Sucre

 Datos actualizados cada 1-5 minutos
(dependiendo de cobertura)

...

****NO es monitoreo instantáneo**** (no es Netflix 😊)

****ES monitoreo supervisado**** con alertas y tendencias

🗝️ ESTRATEGIAS PARA PÉRDIDA DE CONEXIÓN

****Protocolo de Escalamiento****

...

Nivel 1: DETECCIÓN LOCAL (0ms delay)

- └ IA procesa → Detecta somnolencia
- └ Alarma INMEDIATA en cabina
- └ Guarda en buffer local

Nivel 2: SYNC CUANDO HAY RED (1-5 min delay)

- └ Envía resumen al cloud
- └ Admin ve notificación
- └ Puede llamar al chofer

Nivel 3: SIN CONEXIÓN PROLONGADA (>30 min)

- └ Edge device sigue funcionando 100%
- └ Alarmas locales siguen activas
- └ Buffer local guarda todo
- └ Admin ve "última posición conocida"
- └ Al recuperar conexión: Sync batch de datos

Nivel 4: EMERGENCIA

- └ Si chofer no responde a alertas locales
- └ Sistema puede activar:
 - └ Luces de emergencia (si integrado)
 - └ Reducción gradual de velocidad (IA avanzada)
 - └ Llamada automática a contacto de emergencia

...

💡 RESPUESTAS A TUS PREGUNTAS ESPECÍFICAS

¿Está bien que sea una aplicación web?

Respuesta: ❌ **NO para el chofer**, ✅ **Sí para el admin**

...

CHOFER (en vehículo) ❌ NO: App web pura (navegador) Razón: Depende de internet ✅ Sí: Una de estas opciones: 1. App web (con procesamiento offline) 2. App nativa Android/iOS (con procesamiento offline) 3. PWA (Progressive Web App) con Service Workers para offline	
ADMIN (en oficina) ✅ Sí: App web (tu implementación actual) Razón: Siempre tiene internet estable	

...

¿Servidor en la nube o local?

****Respuesta:** 🚧 **HÍBRIDO** (la única opción viable)**

...

Edge Device (Vehículo)

- └ FastAPI local (puerto 8000)
- └ MediaPipe procesando frames
- └ SQLite guardando datos
- └ React UI (localhost:3000)

↕ (sync periódico)

Cloud (AWS/DigitalOcean)

- └ PostgreSQL (datos históricos)
- └ Dashboard Admin (Next.js/React)
- └ API para consultas y reportes

...

**¿Qué pasa si el servidor cloud se cae?*

****Respuesta:** ✅ **El vehículo sigue funcionando perfecto****

...

Impacto de Cloud Down:

- └ Detección IA: ✅ Sigue funcionando (local)
- └ Alertas al chofer: ✅ Siguen funcionando (local)
- └ Guardar datos: ✅ Buffer local crece
- └ Dashboard admin: ❌ No disponible temporalmente
- └ GPS tracking: ❌ No se actualiza en mapa

Recuperación:

- └ Al volver cloud: Sync automático del buffer

Ventajas del cambio:

1. ✅ Funciona en el mundo real (carreteras sin señal)
2. ✅ Alertas instantáneas (salva vidas)
3. ✅ Costos operativos manejables
4. ✅ Escalable a flotas grandes
5. ✅ Cumple regulaciones de seguridad
6. ✅ Tu código actual se aprovecha 80%

🔍 ANÁLISIS CRÍTICO DEL SISTEMA - Ingeniero de Software Senior

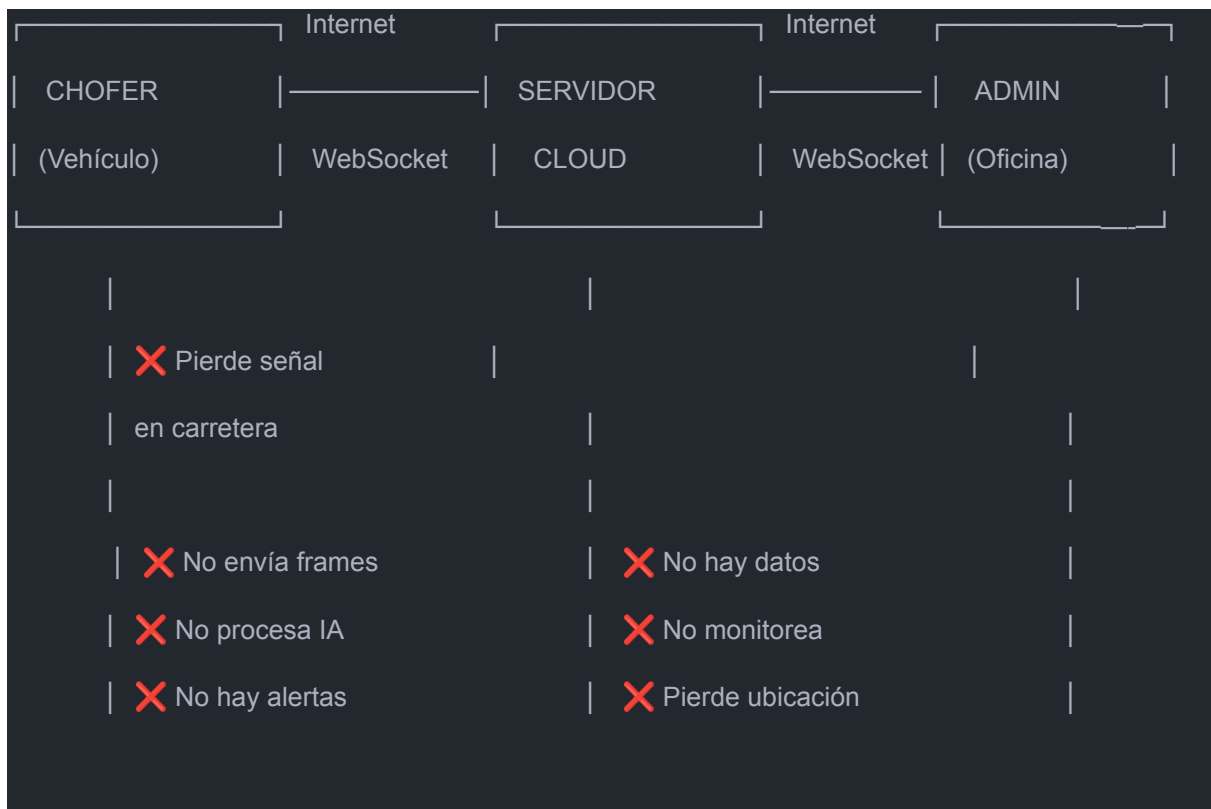
He analizado profundamente tu sistema y las observaciones recibidas. Voy a ser **muy crítico y realista** sobre los desafíos técnicos que enfrentas. Este análisis abordará:

1. ✅ **Problemas críticos identificados**
2. ⚠️ **Escenarios "Qué pasa si..." adicionales**
3. 🛠️ **Soluciones arquitectónicas viables**
4. 📊 **Comparativa: App Web vs App Nativa vs Híbrida**
5. 🎯 **Propuesta de arquitectura final**

🚨 PARTE 1: PROBLEMAS CRÍTICOS IDENTIFICADOS

1.1 Pérdida de Conexión Chofer ↔ Admin

❌ Situación Actual (CRÍTICA)



⚠️ CONSECUENCIAS GRAVES:

Seguridad comprometida: Si el chofer tiene microsleep en zona sin señal, **NO HAY ALERTA**

Monitoreo inútil: Admin no ve nada durante 30-60 min en zonas sin cobertura

Sistema no cumple su propósito: Prevenir accidentes requiere 100% disponibilidad

1.2 Dependencia Total de Internet

✗ Problemas con App Web Pura

```
// Escenario actual (INVIABLE)
const MonitoreoChofer = () => {
  const ws = useWebSocket('wss://servidor-cloud.com/ws');
  // ✗ Si no hay internet:
  // 1. WebSocket se desconecta
  // 2. No se envían frames a la nube
  // 3. IA en servidor NO procesa
  // 4. NO hay alertas de somnolencia
  // 5. Chofer puede dormirse SIN AVISO
};
```

⚠ FALLOS CRÍTICOS:

Escenario	Consecuencia	Nivel de Riesgo
Sin señal 4G/5G en carretera	✗ Sistema completamente inoperativo	● CRÍTICO
Servidor cloud caído (AWS/Azure down)	✗ Todos los choferes sin protección	● CRÍTICO
Latencia alta (>500ms)	✗ Alertas llegan tarde (accidente)	● CRÍTICO
Cuota de datos agotada	✗ Conexión cortada	● ALTO
Túnel/zona rural	✗ Sin servicio 5-30 min	● CRÍTICO

1.3 Procesamiento IA en la Nube

✗ Limitaciones Técnicas

```
# Flujo actual (PROBLEMÁTICO)
# 1. Cámara captura frame (1920x1080) → ~500 KB
# 2. Envía a servidor cloud vía WebSocket
# 3. Servidor procesa con MediaPipe (100-200ms)
# 4. Retorna resultado JSON + imágenes

# ✗ PROBLEMAS:
# - Latencia total: 300-1000ms (inaceptable para alertas)
# - Consumo datos: ~15 MB/min → 900 MB/hora
```

- Sin internet: 0% funcionalidad

 PARTE 3: SOLUCIONES ARQUITECTÓNICAS VIABLES

3.1 SOLUCIÓN HÍBRIDA: Edge Computing + Cloud Sync

 Arquitectura Propuesta



 Análisis histórico + ML para patrones	
 Notificaciones email/SMS a admin	

 Flujo de Funcionamiento

```
# DISPOSITIVO LOCAL EN VEHÍCULO (Offline-First)

# 1. Captura de video
camara.capturar_frame() → frame_local

# 2. Procesamiento LOCAL (SIN INTERNET)
mediapipe_local.procesar(frame_local) → landmarks
ia_local.detectar_somnolencia(landmarks) → {
    "microsueno": True,
    "duracion": 3.2,
    "timestamp": "2025-11-10 14:23:45",
    "ubicacion_gps": "-12.0464, -77.0428"
}

# 3. Alerta INMEDIATA (sin depender de servidor)
if microsueno:
    alarma_sonora.reproducir() #  50ms latencia

# 4. Guardar en SQLite local
db_local.insert(evento_somnolencia)

# 5. Sincronizar cuando HAY internet (Background task)
if internet_disponible():
    cola_mensajes.enviar(eventos_pendientes) # Async
    admin_websocket.enviar(estado_actual) # Real-time
```

 VENTAJAS DE ESTA ARQUITECTURA

Característica	Beneficio
Procesamiento local	Alertas en 50ms, sin depender de internet
SQLite local	Datos guardados aunque servidor caiga

Sincronización asíncrona	Admin ve datos cuando hay señal
GPS offline	Ubicación se guarda localmente
Modelo IA ligero	TensorFlow Lite/ONNX (20 MB vs 500 MB)

3.2 COMPARATIVA: App Web vs App Nativa vs Híbrida

Criterio	App Web Pura 	App Nativa 
Procesamiento IA sin internet	✗ Imposible	✓ Sí (TensorFlow Lite)
Acceso a cámara	⚠ Requiere HTTPS + permisos	✓ Nativo
GPS offline	✗ No	✓ Sí
Base de datos local	⚠ IndexedDB (5-50 MB límite)	✓ SQLite ilimitado
Instalación	✓ URL en navegador	✗ Tienda App Store/Play Store
Actualizaciones	✓ Automáticas	✗ Manual usuario
Compatibilidad dispositivos	✓ Cualquier tablet/PC	✗ Solo iOS/Android
Costo desarrollo	● Bajo (1 codebase)	● Alto (iOS + Android)
Rendimiento IA	● Bajo (depende servidor)	● Alto (GPU local)
Latencia alertas	● 300-1000ms	● 30-50ms
Recomendación	✗ NO VIABLE para seguridad crítica	✓ Viable pero costoso

DOCUMENTACIÓN: SOLUCIÓN HÍBRIDA Edge Computing + Cloud Sync

Visión General

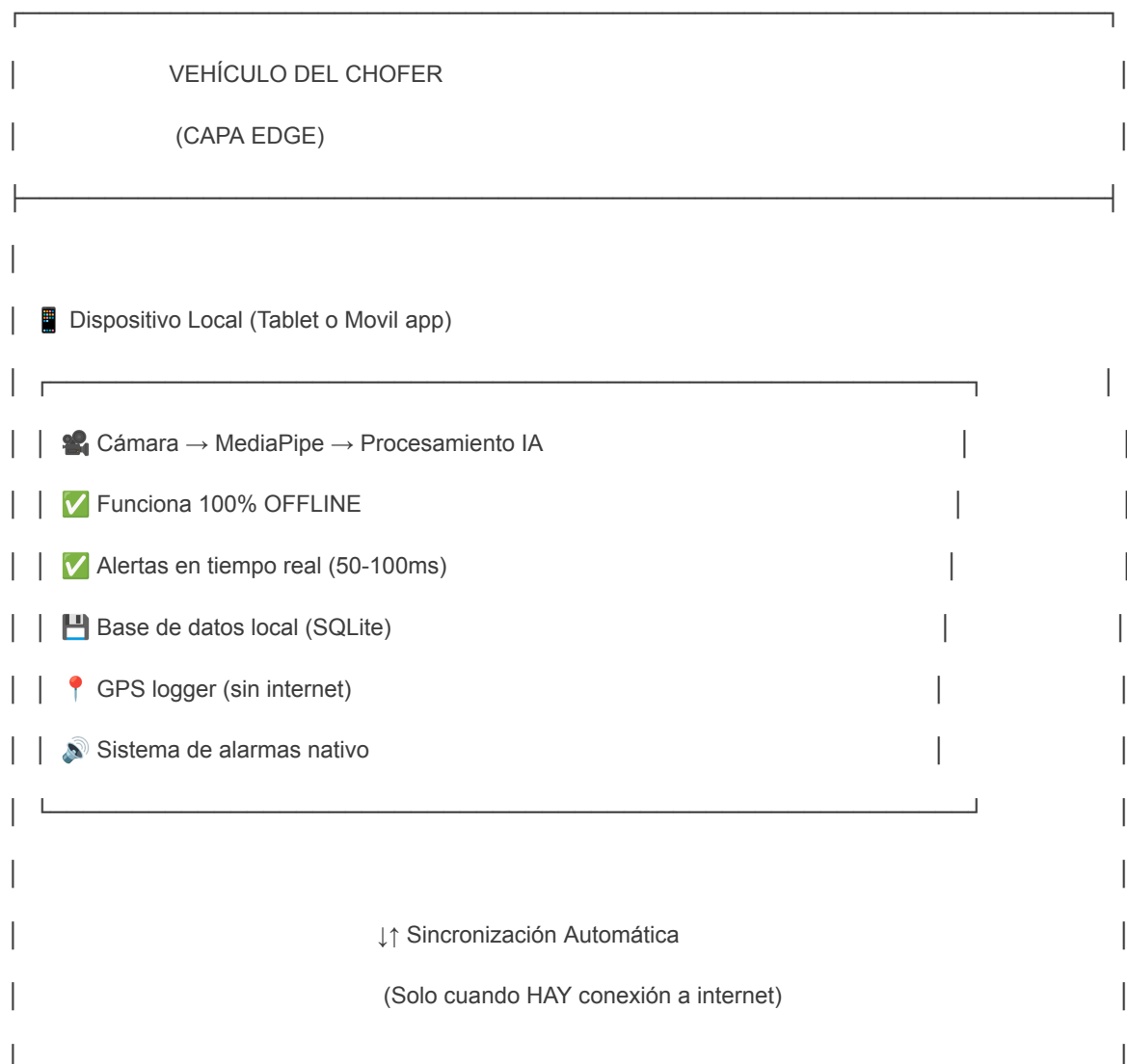
La **Solución Híbrida Edge Computing + Cloud Sync** es una arquitectura de software que distribuye el procesamiento entre **dos capas principales**:

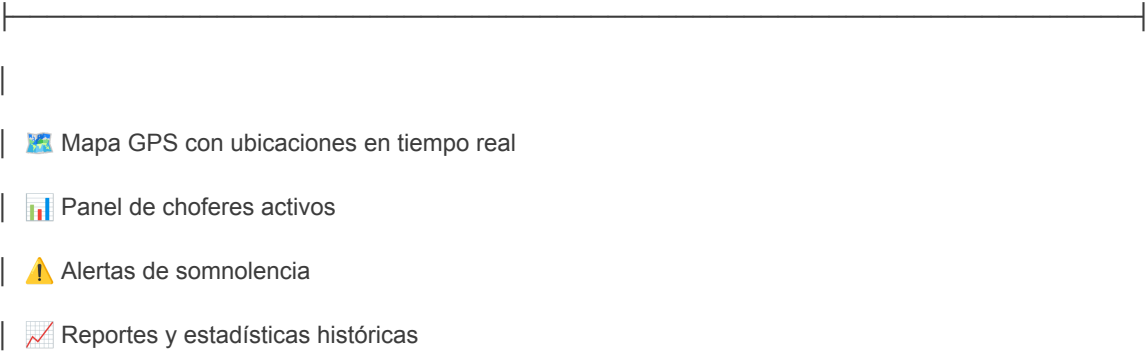
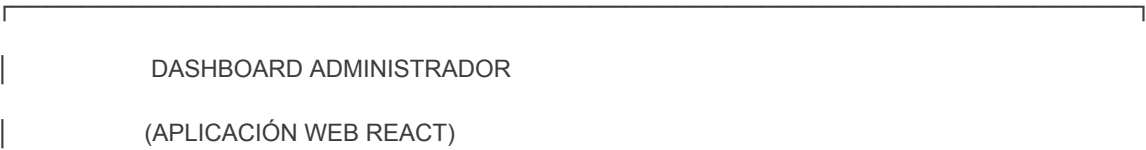
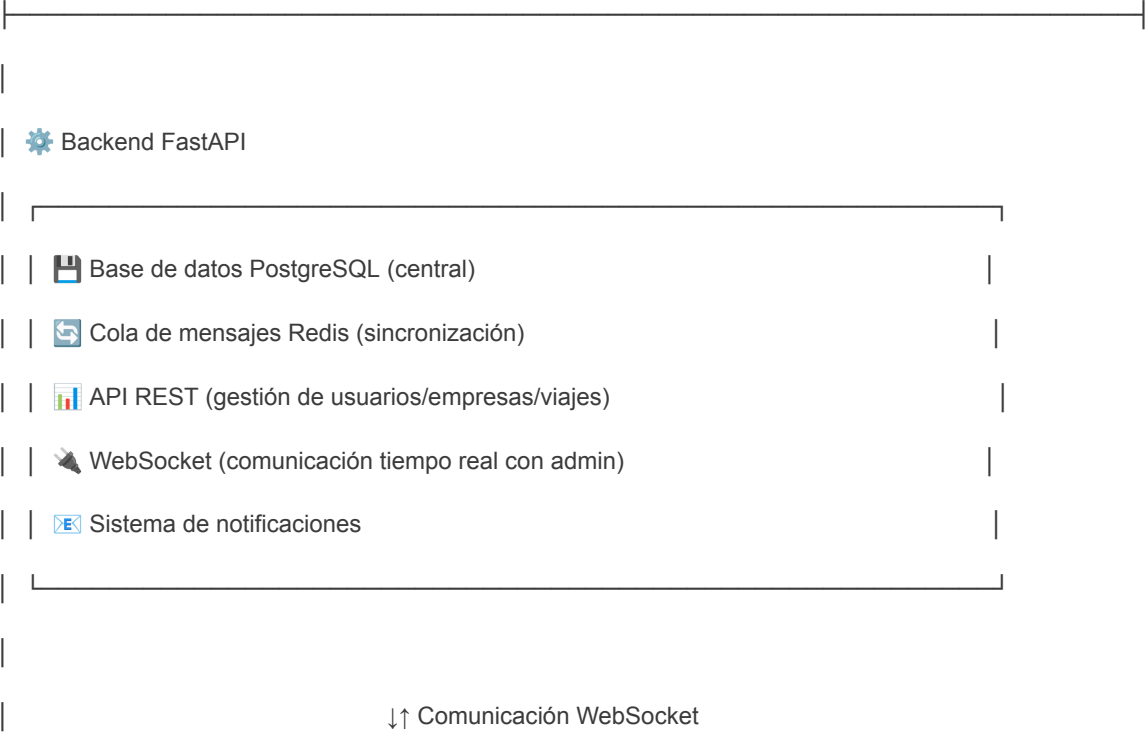
1. **EDGE (Borde/Local)**: Procesamiento en el dispositivo del vehículo (offline-first)
2. **CLOUD (Nube)**: Almacenamiento centralizado y monitoreo administrativo

Esta arquitectura garantiza que el sistema **funcione siempre**, incluso sin conexión a internet, mientras mantiene capacidades de monitoreo remoto cuando hay conectividad disponible.

Arquitectura del Sistema

Componentes Principales





Funcionamiento del Sistema

1. Procesamiento Local (EDGE) - Sin Internet

¿Qué ocurre en el vehículo del chofer?

El dispositivo instalado en el vehículo (tablet app móvil) ejecuta **todas las funciones críticas de seguridad de forma local**:

Captura de Video

- La cámara del dispositivo graba continuamente al conductor
- Los frames de video se procesan localmente en el dispositivo
- No se envía video a internet (ahorro de datos y privacidad)

Procesamiento con Inteligencia Artificial

- **MediaPipe** extrae 468 puntos faciales y 21 puntos por mano
- El modelo de IA analiza estos puntos para detectar:
 - Microsueños (ojos cerrados >2 segundos)
 - Bostezos (boca abierta >4 segundos)
 - Inclínación de cabeza (cabeceo >3 segundos)
 - Frotamiento de ojos (mano cerca del rostro)
 - Parpadeo excesivo (>20 parpadeos/minuto)

Alertas Inmediatas

- Cuando se detecta somnolencia, el sistema:
 - Reproduce alarma sonora de alta intensidad
 - Muestra notificación visual en pantalla
 - Vibra el dispositivo (si es tablet)
- **Latencia total: 50-100 milisegundos** (casi instantáneo)
- **Funciona incluso sin internet**

Almacenamiento Local

- Cada evento de somnolencia se guarda en una base de datos **SQLite local**:
 - Timestamp exacto del evento
 - Tipo de somnolencia detectada
 - Duración en segundos
 - Coordenadas GPS del vehículo en ese momento
 - Estado de sincronización (pendiente/completado)

Registro GPS Offline

- El GPS del dispositivo registra la ubicación cada 30 segundos
- Se guarda localmente aunque no haya internet
- Permite reconstruir la ruta completa del viaje después

2. Sincronización Inteligente (EDGE → CLOUD)

¿Cómo se comunican el vehículo y el servidor?

La sincronización ocurre **automáticamente en segundo plano** cuando el dispositivo detecta conexión a internet:

Detección de Conectividad

- El sistema verifica cada 10 segundos si hay conexión a internet
- Cuando detecta señal (WiFi, 4G, 5G):
 - Inicia proceso de sincronización automática
 - No interrumpe el monitoreo en curso

Envío de Datos Acumulados

- El dispositivo consulta la base SQLite local
- Selecciona todos los eventos marcados como "no sincronizados"
- Los empaqueta en formato JSON comprimido
- Los envía al servidor en lotes (batches) para optimizar ancho de banda

Ejemplo de datos enviados:

```
{
  "id_chofer": 123,
  "eventos": [
    {
      "timestamp": "2025-11-10 14:23:45",
      "tipo": "microsueno",
      "duracion_segundos": 3.2,
      "latitud": -12.0464,
      "longitud": -77.0428
    },
    {
      "timestamp": "2025-11-10 14:45:12",
      "tipo": "bostezo",
      "duracion_segundos": 5.1,
      "latitud": -12.0512,
      "longitud": -77.0356
    }
  ]
  // ... más eventos acumulados
}
```


✓ Confirmación y Marcado

- El servidor recibe los eventos y los guarda en PostgreSQL
- Responde con confirmación: `{"status": "ok", "eventos_guardados": 15}`
- El dispositivo marca esos eventos como "sincronizados" en SQLite
- Los eventos sincronizados se conservan localmente para histórico

↺ Manejo de Fallos

- Si la sincronización falla (servidor caído, internet interrumpido):
 - Los eventos permanecen como "no sincronizados"
 - El sistema reintentará en el próximo ciclo (30 segundos después)
 - No se pierde ningún dato

3. Monitoreo en Tiempo Real (CLOUD → ADMIN)

¿Cómo el administrador monitorea a los choferes?

El dashboard web del administrador recibe información en tiempo real **solo cuando los choferes tienen conexión a internet**:

📡 Conexión WebSocket Bidireccional

- Cuando el chofer tiene internet, su dispositivo abre un canal WebSocket con el servidor
- El servidor mantiene una lista de "choferes actualmente conectados"
- El dashboard del admin se suscribe a actualizaciones de todos los choferes

📊 Información en Tiempo Real

- **Ubicación GPS**: Mapa muestra la posición actual de cada chofer
- **Estado de somnolencia**: Indicadores visuales (verde/amarillo/rojo)
- **Última alerta**: Cuándo y qué tipo de somnolencia se detectó
- **Estadísticas del viaje**: Total de microsueños, bostezos, etc.

🗺 Visualización en Mapa

- Marcadores por chofer con colores según estado:
 -  Verde: Sin alertas recientes
 -  Amarillo: Alerta leve (1-2 eventos en última hora)
 -  Rojo: Alerta crítica (3+ eventos o microsueño prolongado)
- Click en marcador muestra detalles del chofer
- Ruta del viaje trazada en el mapa

⚠ Notificaciones Proactivas

- Cuando un chofer tiene evento crítico de somnolencia:

- El admin recibe notificación push en su dashboard
- Se envía email/SMS automático al supervisor
- Se registra en bitácora de alertas

Manejo de Desconexiones

- Si un chofer pierde señal:
 - Su marcador en el mapa se pone gris
 - Se muestra "Última conexión: hace 5 minutos"
 - El admin ve la última ubicación conocida
 - Cuando recupera señal, todos los eventos acumulados llegan en lote

Ventajas de Esta Arquitectura

Seguridad y Confiabilidad

Escenario	Sin Arquitectura Híbrida	Con Arquitectura Híbrida
Sin señal en carretera	✗ Sistema no funciona, no hay alertas	✓ Alertas locales funcionan normalmente
Servidor caído	✗ Ningún chofer protegido	✓ Todos los choferes protegidos, solo admin sin visibilidad
Internet lento	✗ Latencia alta, alertas tardías	✓ Alertas inmediatas (procesamiento local)
Consumo de datos	✗ 3.6 GB/hora (video streaming)	✓ ~5 MB/hora (solo JSON con eventos)
Pérdida de datos	✗ Eventos no registrados si hay falla	✓ SQLite local garantiza 0% pérdida

Componentes Técnicos Clave

En el Dispositivo del Vehículo (EDGE)

Aplicación Móvil

- Kotlin
- Se instala como programa nativo en Windows/Mac/Linux
- Acceso completo a hardware: cámara, GPS, almacenamiento, notificaciones

MediaPipe (Biblioteca de Google)

- Detecta 468 puntos faciales en tiempo real
- Procesa 20-30 frames por segundo
- Funciona offline (modelo incluido en la aplicación)

SQLite

- Base de datos embebida (archivo único)
- No requiere servidor externo
- Perfecta para almacenamiento local

APIs Nativas del Sistema Operativo

- Acceso a GPS (geolocalización)
- Reproducción de sonidos (alarmas)
- Notificaciones del sistema
- Vibración (en tablets)

En el Servidor Cloud (CLOUD)

FastAPI (Backend)

- Tu código actual se mantiene
- Se agregan endpoints de sincronización
- WebSocket para comunicación admin-servidor

PostgreSQL

- Base de datos central que almacena:
 - Usuarios (admins y choferes)
 - Empresas de transporte
 - Viajes asignados
 - Eventos de somnolencia sincronizados
 - Histórico completo

Redis

- Cola de mensajes para sincronización asíncrona
- Caché para mejorar rendimiento
- Gestión de sesiones WebSocket

Sistema de Notificaciones

- Envío de emails (SMTP)
- SMS (Twilio/similar)
- Push notifications (Firebase)

En el Dashboard Admin (WEB)

Aplicación Web React

- Tu código actual se mantiene
- Se agrega componente de mapa GPS
- Panel de choferes en tiempo real

Mapbox/Leaflet

- Biblioteca para mostrar mapas interactivos
- Marcadores personalizados por chofer
- Trazado de rutas

WebSocket Client

- Conexión permanente con el servidor
- Recibe actualizaciones en tiempo real
- Notificaciones instantáneas



Flujo Completo de un Evento de Somnolencia

Caso 1: Chofer CON Conexión a Internet

1. Chofer conduce por autopista (señal 4G disponible)



2. Dispositivo procesa video localmente (MediaPipe)



3. IA detecta microsueño de 3.2 segundos



4. INMEDIATAMENTE:

- Alarma sonora se reproduce (50ms latencia)
- Notificación en pantalla del dispositivo



5. Evento se guarda en SQLite local con timestamp y GPS



6. 2 segundos después:

- Sincronización automática envía evento al servidor
- Servidor guarda en PostgreSQL

↓

7. 1 segundo después:

- WebSocket notifica al dashboard admin
- Admin ve alerta en tiempo real en el mapa
- Admin puede llamar al chofer si es necesario

Tiempo total desde detección hasta alerta admin: ~3 segundos

Caso 2: Chofer SIN Conexión a Internet

1. Chofer conduce por zona rural sin señal

↓

2. Dispositivo procesa video localmente (MediaPipe)

↓

3. IA detecta microsueño de 4.1 segundos

↓

4. INMEDIATAMENTE:

- Alarma sonora se reproduce (50ms latencia)
- Notificación en pantalla del dispositivo

↓

5. Evento se guarda en SQLite local con timestamp y GPS

↓

6. Sincronización detecta que NO HAY internet

- Marca evento como "pendiente de sincronizar"
- Sistema sigue funcionando normalmente

↓

7. 30 minutos después, chofer llega a zona con señal

↓

8. Sincronización automática detecta internet

- Envía los 12 eventos acumulados en lote
- Servidor los guarda en PostgreSQL

↓

9. Admin recibe notificación:

"Chofer Juan Pérez estuvo sin señal 30 min.
Se detectaron 12 eventos de somnolencia.
Revisar histórico."

Resultado: El chofer estuvo SIEMPRE protegido, el admin solo perdió visibilidad temporal

Beneficios Clave de la Solución

Para el Chofer

- ✓ **Protección 24/7:** Alertas funcionan siempre, con o sin internet
- ✓ **Sin interrupciones:** No depende de señal para funcionar

Para el Administrador

- ✓ **Visibilidad total:** Ve ubicación y estado de todos los choferes (cuando hay señal)
- ✓ **Histórico completo:** Ningún evento se pierde, todos se sincronizan eventualmente
- ✓ **Notificaciones críticas:** Alertas inmediatas cuando chofer en peligro
- ✓ **Escalable:** Puede monitorear flotas grandes sin sobrecargar servidores

Conclusión

La **Solución Híbrida Edge Computing + Cloud Sync** es la única arquitectura viable para un sistema de detección de somnolencia crítico de seguridad porque:

1. **Garantiza alertas siempre** (independiente de conectividad)
2. **Optimiza recursos** (bajo consumo de datos y servidores)
3. **Cumple regulaciones** (privacidad y trazabilidad)
4. **Escala eficientemente** (soporta flotas grandes)
5. **Balancea necesidades** (seguridad local + monitoreo remoto)

Esta arquitectura transforma un sistema que **podría fallar en momentos críticos** en uno que **nunca falla en su función principal: proteger al conductor**.

Documentación creada para: **Sistema de Detección de Somnolencia en Tiempo Real**

DOCUMENTACIÓN: ARQUITECTURA HÍBRIDA Edge Computing + Cloud Sync

Visión General

La **Arquitectura Híbrida Edge Computing + Cloud Sync** es un diseño de sistema que distribuye la responsabilidad del procesamiento en **dos capas independientes pero complementarias**:

1. **EDGE (Borde/Local)**: Procesamiento crítico en el dispositivo del vehículo (funciona offline)
2. **CLOUD (Nube)**: Almacenamiento centralizado y monitoreo administrativo (requiere internet)

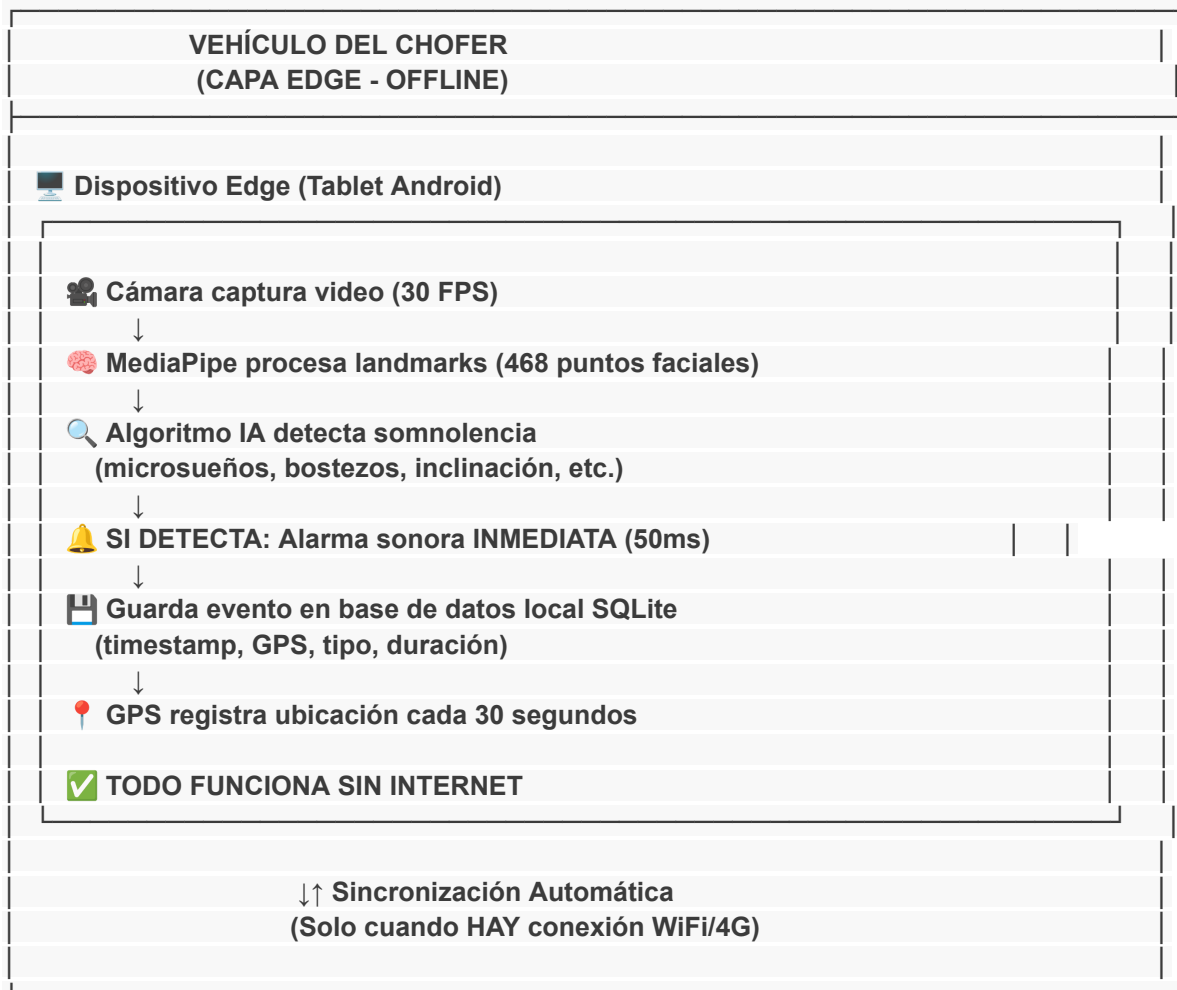
Principio Fundamental

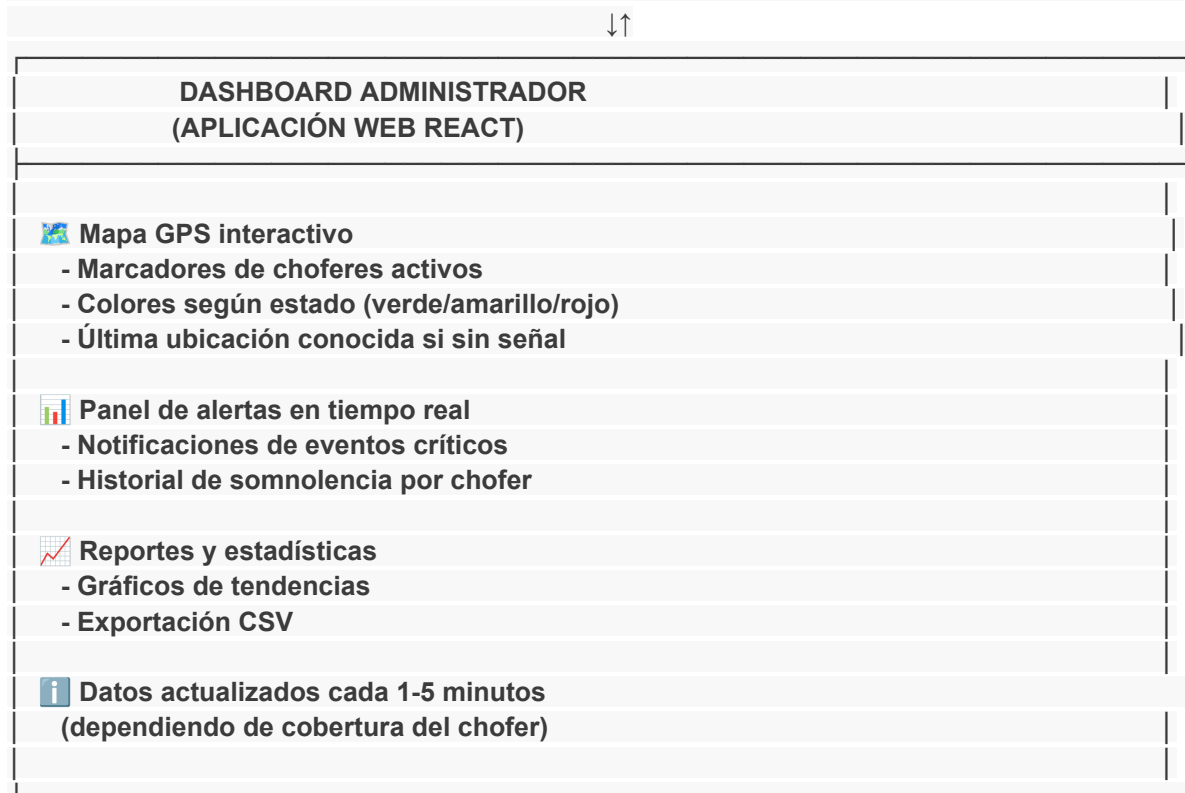
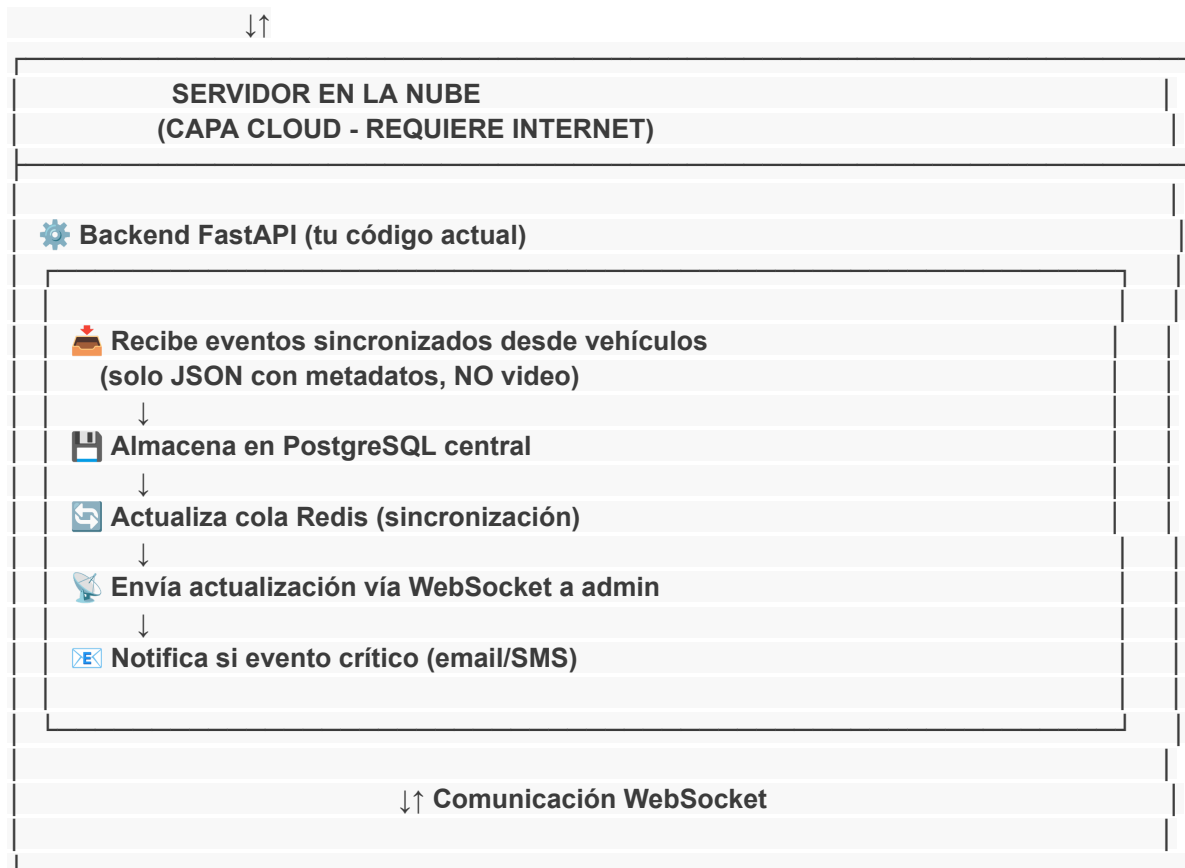
 **CRÍTICO (Edge)**: Detección de somnolencia + Alertas

 **IMPORTANTE (Cloud)**: Monitoreo remoto + Histórico

La **seguridad del chofer NUNCA depende de internet**. El admin puede perder visibilidad temporal, pero el conductor siempre está protegido.

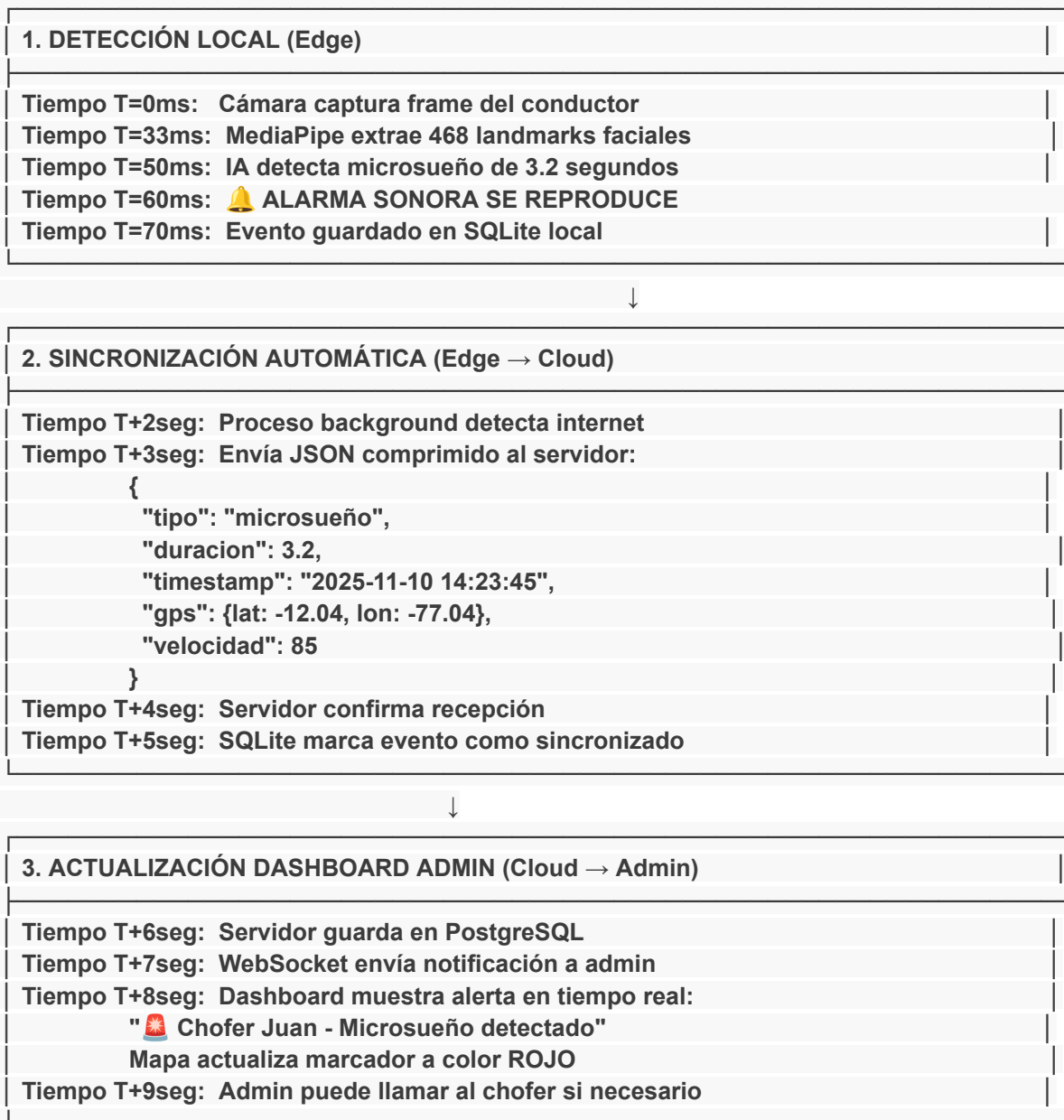
Arquitectura Completa del Sistema



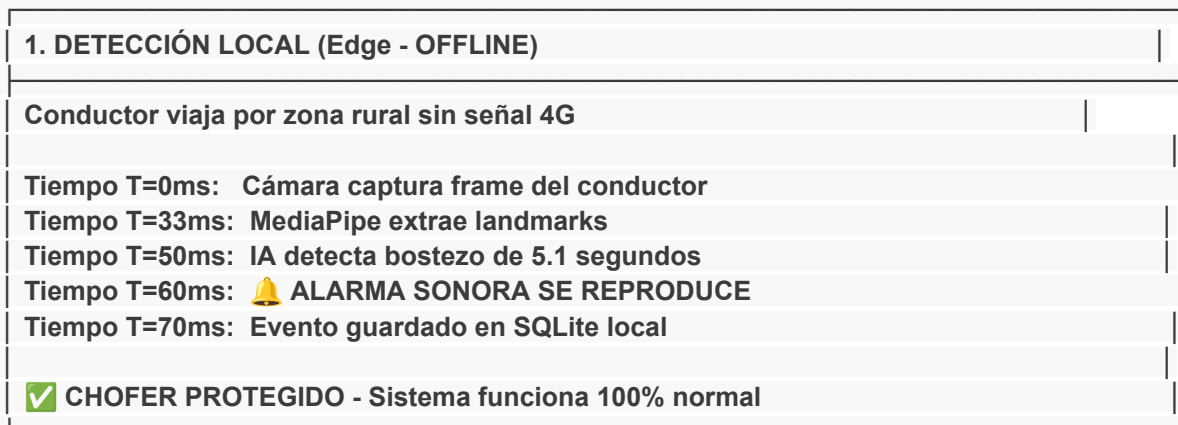


Flujo de Funcionamiento Completo

ESCENARIO 1: Chofer CON Conexión a Internet



ESCENARIO 2: Chofer SIN Conexión a Internet





2. BUFFER LOCAL ACUMULA EVENTOS (Edge)

Durante 45 minutos sin señal, el sistema:

- Detecta 3 bostezos → Alarmas reproducidas ✓
- Detecta 1 microsueño → Alarma reproducida ✓
- Detecta 2 inclinaciones → Alarmas reproducidas ✓

Todos los eventos guardados en SQLite local:

- 6 eventos totales con timestamps y GPS
- Estado: "pendiente_sincronizar"
- Tamaño total: ~8 KB (JSON)

- ⚠ ADMIN NO VE NADA (sin conexión del chofer)
- ✓ CHOFER SIGUE PROTEGIDO (alarmas locales activas)



3. RECUPERACIÓN DE SEÑAL (Edge → Cloud)

Minuto 45: Chofer llega a zona con cobertura 4G

Tiempo T=0seg: Sistema detecta conexión a internet

Tiempo T=2seg: Lee 6 eventos pendientes de SQLite

Tiempo T=3seg: Envía lote (batch) al servidor:
[evento1, evento2, ..., evento6]

Tiempo T=5seg: Servidor recibe y procesa los 6 eventos

Tiempo T=6seg: Servidor confirma: "6 eventos guardados"

Tiempo T=7seg: SQLite marca los 6 como sincronizados



4. ADMIN RECIBE HISTÓRICO (Cloud → Admin)

Tiempo T=8seg: Dashboard admin recibe notificación:

🔊 "Chofer Juan reconectado después de 45 min sin señal"

⚠ "Se detectaron 6 eventos de somnolencia:"

- 3 bostezos (14:15, 14:28, 14:42)
- 1 microsueño (14:35)
- 2 inclinaciones (14:20, 14:50)

Mapa actualiza ruta completa con puntos GPS históricos

Admin puede revisar timeline completo del viaje

Admin puede llamar al chofer para verificar estado

Resultado: El chofer estuvo **siempre protegido**. El admin solo perdió visibilidad temporal, pero recupera el histórico completo al reconectar.

OPCIÓN : Implementación con Aplicación Android Nativa

1. Hardware Necesario

DISPOSITIVO ANDROID EN EL VEHÍCULO
 Tablet Android ($\geq$ Android 10) - Mínimo: 4GB RAM, procesador octacore - Recomendado: Samsung Galaxy Tab A9+ - Pantalla: 8-10 pulgadas - Cámara frontal: 8MP o superior - GPS integrado - Batería: 5000+ mAh
 Cargador vehicular 12V USB-C (2.4A) - Alimentación continua durante viaje
 Soporte de tablero antivibración - Montaje con ventosa o adhesivo - Ángulo ajustable
 Altavoz Bluetooth (opcional) - Para alarma más potente que speaker tablet

2. Arquitectura de la Aplicación Android

APP NATIVA ANDROID (Kotlin + Jetpack Compose)
 CAPAS DE LA APLICACIÓN:
1. CAPA DE PRESENTACIÓN (UI) - Jetpack Compose (UI declarativa) - ViewModel (estado de la app) - Navigation Component (pantallas)
2. CAPA DE DOMINIO (Lógica IA) - MediaPipe Face Landmarker (nativo Android) - Algoritmos de detección de somnolencia - TensorFlow Lite (modelo custom opcional)
3. CAPA DE DATOS - Room Database (SQLite local) - DataStore (preferencias) - Retrofit (sincronización HTTP)
4. SERVICIOS EN BACKGROUND - CameraX (captura video)

- WorkManager (sincronización periódica)
- ForegroundService (detección activa)
- LocationManager (GPS)

3. Funcionamiento de la App Android

Inicio de Viaje

1. AUTENTICACIÓN

- Chofer abre app en tablet
- Login con credenciales (ID + contraseña)
- App valida con servidor (requiere internet inicial)
- Servidor retorna token JWT

2. INICIAR MONITOREO

- Chofer presiona "Comenzar Viaje"
- App solicita permisos:
 - * Cámara
 - * Ubicación (GPS)
 - * Audio (alarma)
 - * Notificaciones
- App crea registro en Room Database:
INSERT INTO viajes (id_chofer, inicio, estado)

3. PROCESAMIENTO EN FOREGROUND SERVICE

- Service se ejecuta aunque app minimizada
- Muestra notificación persistente:
"🚗 Sistema de detección activo"
- No se puede cerrar sin detener viaje

Ciclo de Detección (Loop Principal)

LOOP INFINITO (30 FPS):

1. CameraX captura frame de cámara frontal

- Resolución: 640x480
- Formato: YUV_420_888

2. MediaPipe Face Landmarker procesa

- Detecta rostro
- Extrae 468 landmarks
- Usa GPU del dispositivo (aceleración)

3. Algoritmo de Somnolencia analiza

- Calcula distancias oculares (EAR)
- Mide apertura bucal (MAR)
- Determina ángulo de cabeza

4. SI DETECTA SOMNOLENCIA:

- MediaPlayer reproduce alarma.mp3 (volumen máximo, ignora "no molestar")
- Vibrador activa patrón intenso
- UI muestra alerta visual
- Room Database inserta evento:
INSERT INTO eventos_somnolencia (...)

5. GPS registra ubicación

- LocationManager obtiene coordenadas
- Actualiza cada 30 segundos
- Asocia a evento si hubo alerta

6. Renderizar UI

- Compose dibuja landmarks sobre video
- Actualiza métricas en pantalla

REPETIR (cada 33ms → 30 FPS)

Sincronización en Background

WorkManager (cada 2 minutos):

1. Verificar conectividad

- ConnectivityManager.getActiveNetwork()
- Si no hay WiFi/datos: CANCELAR, reintentar después

2. Consultar eventos no sincronizados

- Room DAO: eventosDao.getNoSincronizados()
- Serializar a JSON

3. Enviar al servidor

- Retrofit POST a /api/v1/sync-events
- Timeout: 30 segundos
- Reintentos: 3 con backoff exponencial

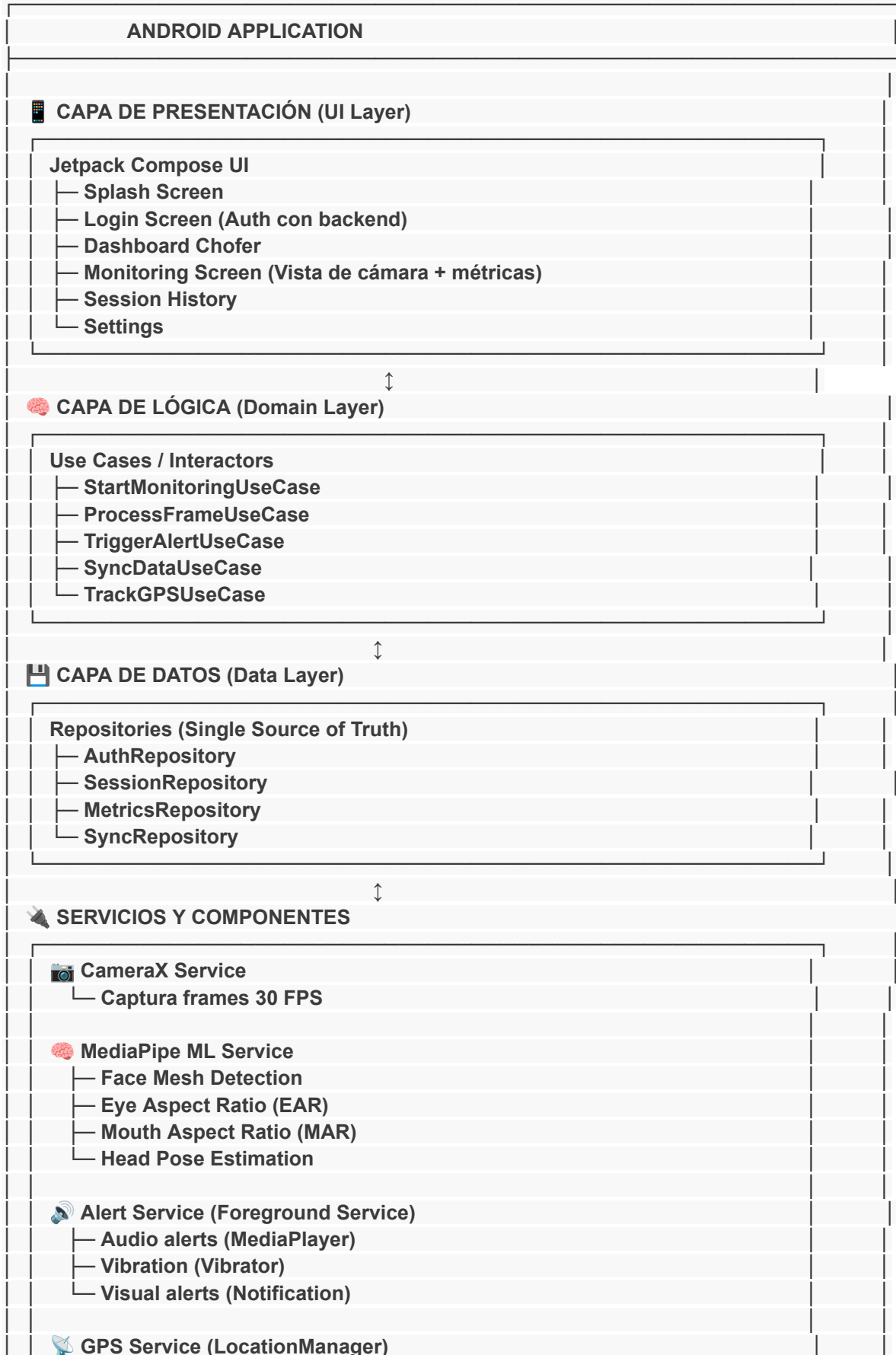
4. Marcar como sincronizados

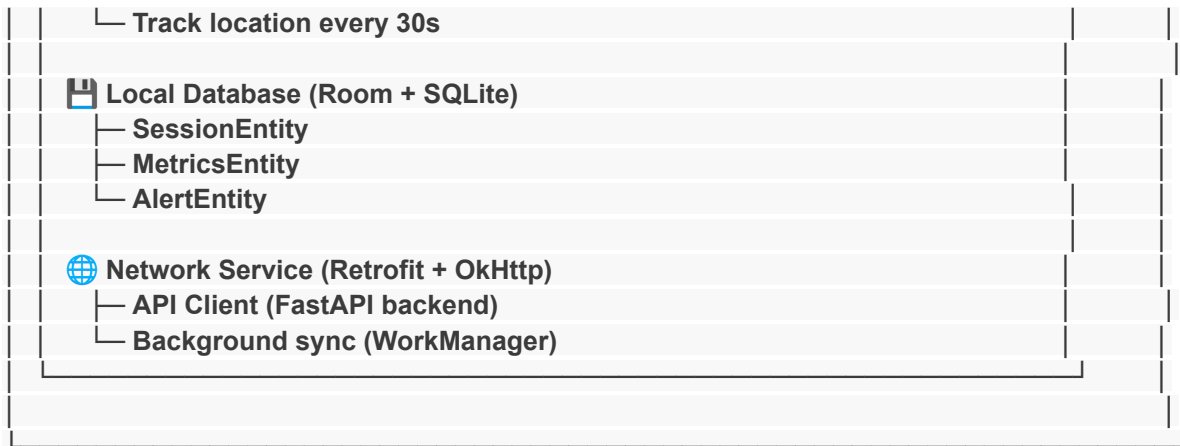
- Room DAO: eventosDao.marcarSincronizados(ids)

5. Notificar al chofer (opcional)

- "✅ Datos sincronizados con servidor"

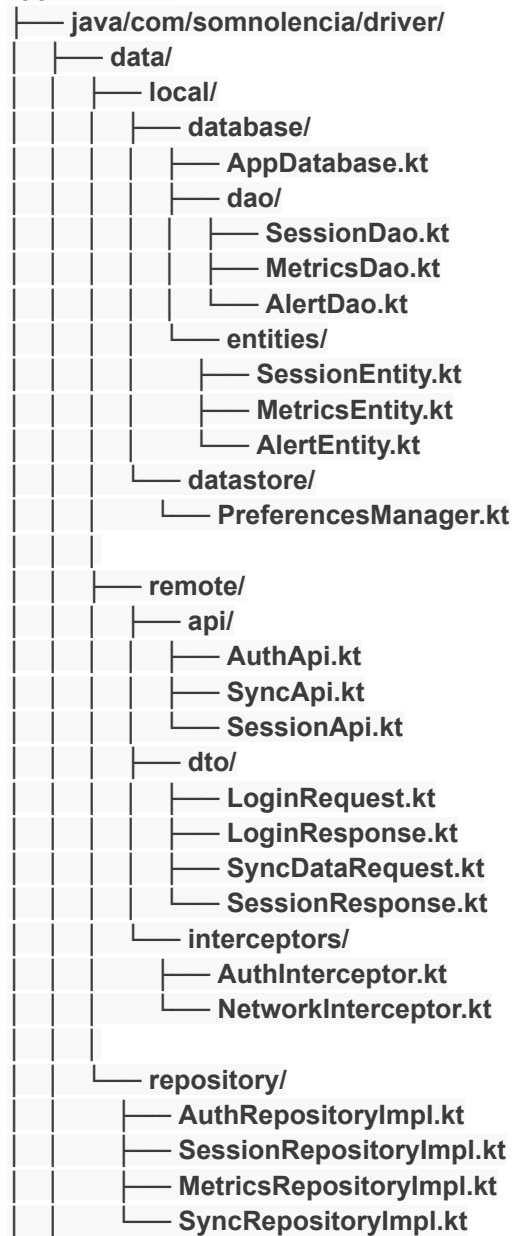
Arquitectura Propuesta - Android App

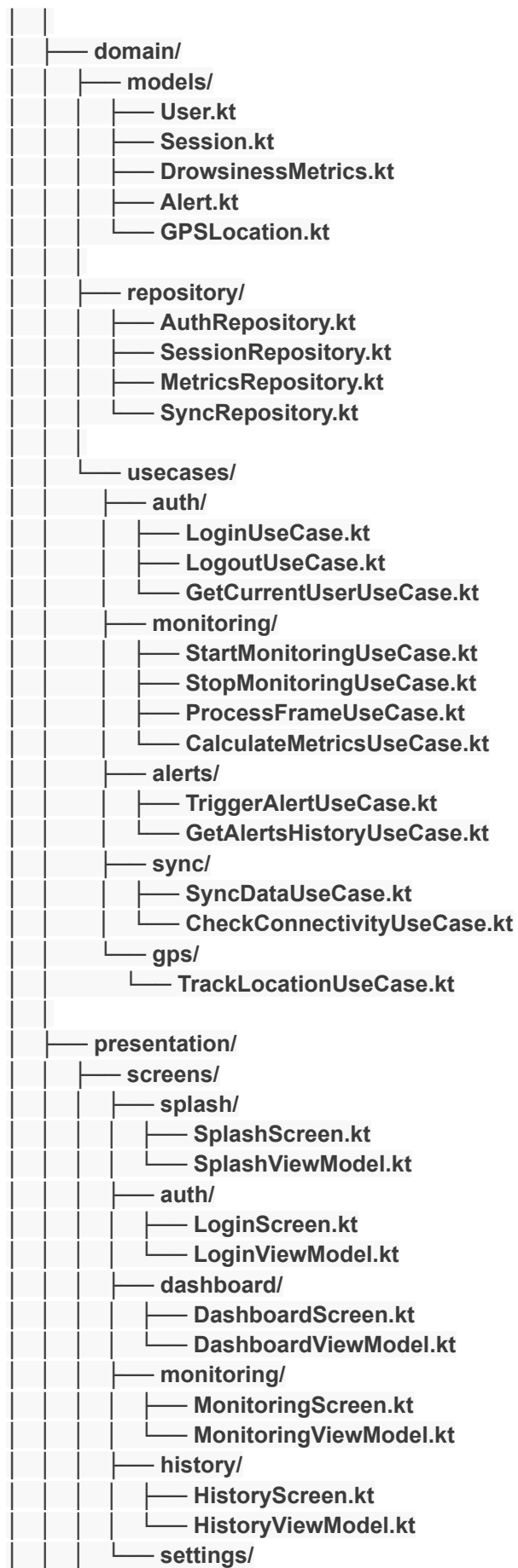


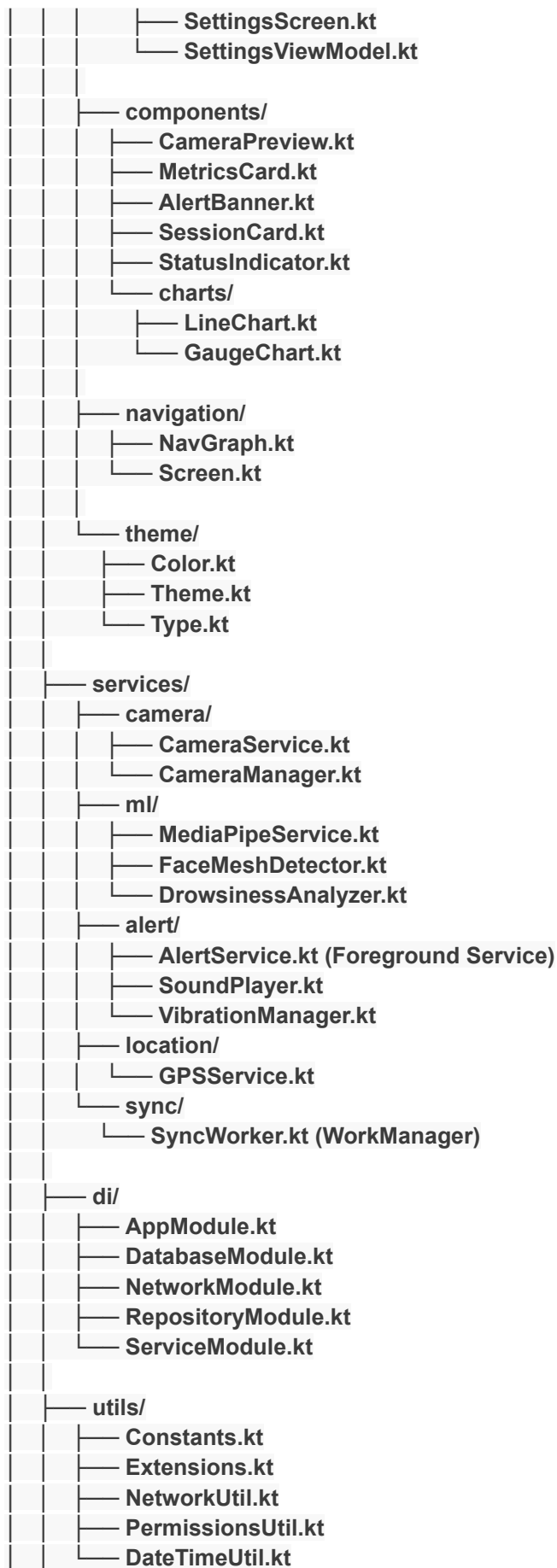


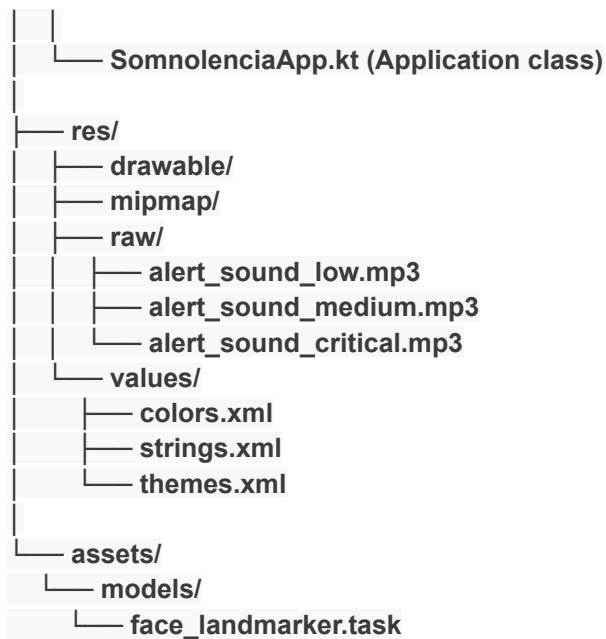
📁 Estructura de Carpetas Detallada

app/src/main/









Roadmap de Implementación (8 Semanas)

Setup y Arquitectura Base

✅ Tareas:

1. Crear proyecto Android Studio (Kotlin + Compose)
2. Configurar dependencias (Hilt, Room, Retrofit)
3. Implementar arquitectura MVVM + Clean
4. Crear entidades de base de datos
5. Configurar Hilt modules
6. Implementar DataStore para preferencias
7. Crear tema Compose (colores, tipografía)

📦 Entregables:

- Proyecto compilable
- Base de datos funcionando
- Navegación básica
- Pantalla de login (UI only)

: Autenticación y API

✅ Tareas:

1. Implementar Retrofit client (integración con FastAPI)
2. Crear AuthRepository + AuthApi
3. Implementar LoginUseCase
4. Conectar LoginViewModel con backend
5. Manejo de tokens JWT
6. Auto-login con tokens guardados
7. Logout functionality

📦 Entregables:

- Login funcional con backend

- Persistencia de sesión
- Manejo de errores de red

: CameraX + Permisos

✓ Tareas:

1. Solicitar permisos de cámara (Accompanist Permissions)
2. Implementar CameraX preview
3. Captura de frames a 30 FPS
4. Conversión de ImageProxy a Bitmap
5. UI de monitoring screen con vista previa
6. Mostrar métricas simuladas en UI

📦 Entregables:

- Cámara funcionando
- Preview en pantalla
- Permisos manejados correctamente

MediaPipe IA + Detección

✓ Tareas:

1. Integrar MediaPipe Face Landmarker
2. Cargar modelo .task desde assets
3. Implementar detección de rostro en frames
4. Calcular EAR (Eye Aspect Ratio)
5. Calcular MAR (Mouth Aspect Ratio)
6. Detectar inclinación de cabeza (Head Pose)
7. Algoritmo de detección de somnolencia
8. Contadores: parpadeos, bostezos, cabeceos

📦 Entregables:

- Detección facial funcionando
- Métricas calculadas en tiempo real
- Algoritmo de somnolencia operativo

Alertas + GPS + Storage

✓ Tareas:

1. Implementar AlertService (Foreground Service)
2. Reproducción de sonidos de alerta
3. Vibración en alertas críticas
4. Notificaciones persistentes
5. Integrar GPS (FusedLocationProvider)
6. Guardar métricas en Room Database
7. Crear workers para limpieza de datos antiguos

📦 Entregables:

- Sistema de alertas funcionando
- GPS tracking activo
- Datos guardados localmente

Sincronización + Testing

✓ Tareas:

1. Implementar SyncWorker (WorkManager)
2. Sync periódico cada 2-5 minutos
3. Retry logic con exponential backoff
4. Manejo de conectividad (online/offline)
5. Pruebas de integración
6. Pruebas de UI (Compose Test)
7. Optimización de rendimiento
8. Documentación de código

📦 Entregables:

- App completa funcionando
- Sync con backend operativo
- Tests pasando
- APK release listo

📖 Recursos y Documentación

Documentación Oficial:

- Android Developers: <https://developer.android.com>
- Jetpack Compose: <https://developer.android.com/jetpack/compose>
- MediaPipe: <https://developers.google.com/mediapipe>
- CameraX: <https://developer.android.com/training/camerax>

Tutoriales Recomendados:

- MediaPipe Face Landmarker Android:
https://developers.google.com/mediapipe/solutions/vision/face_landmarker/android
- Jetpack Compose Tutorial:
<https://developer.android.com/courses/jetpack-compose/course>
- Clean Architecture Android:
<https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>

Repositorios Ejemplo:

- <https://github.com/android/compose-samples>
- <https://github.com/google/mediapipe/tree/master/mediapipe/examples/android>



DOCUMENTACIÓN: APLICACIÓN MÓVIL ANDROID - DETECCIÓN DE SOMNOLENCIA



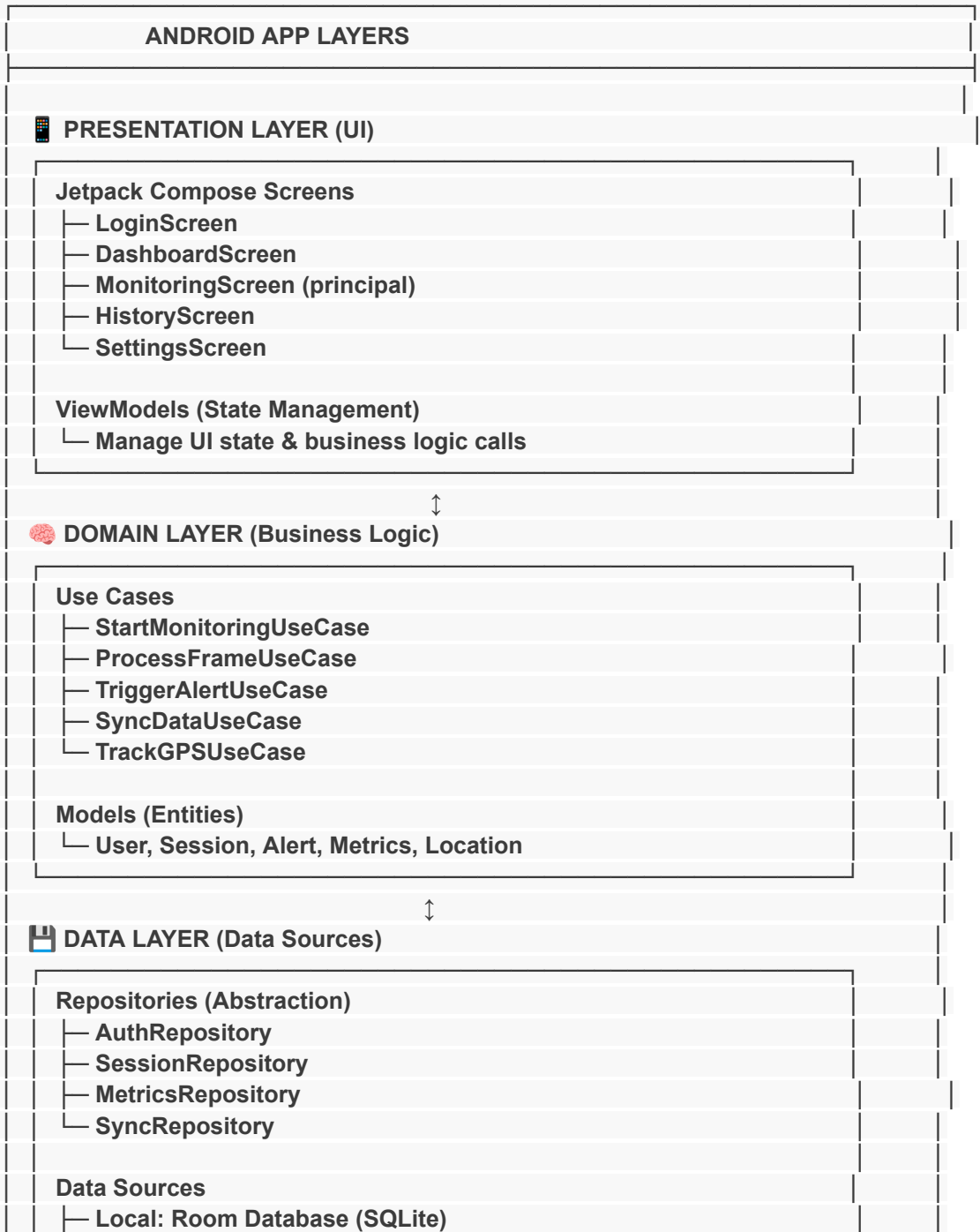
Visión General del Proyecto

Aplicación Android nativa que funciona como **Edge Device** para detectar somnolencia en conductores en tiempo real, con capacidad de operar **100% offline** y sincronización inteligente con el backend cloud cuando hay conectividad.



Arquitectura de la Aplicación

Patrón Arquitectónico: Clean Architecture + MVVM



	Remote: Retrofit API (FastAPI backend)	
	Preferences: DataStore	

Servicios y Componentes Críticos

ANDROID SERVICES		
	CameraX Service	
	└ Captura video frontal 30 FPS	
	MediaPipe ML Service	
	└ Carga modelo Face Landmarker	
	└ Detecta 478 landmarks faciales	
	└ Calcula EAR (Eye Aspect Ratio)	
	└ Calcula MAR (Mouth Aspect Ratio)	
	└ Analiza pose de cabeza	
	Alert Service (Foreground)	
	└ Reproduce alarma sonora	
	└ Activa vibración	
	└ Muestra notificación persistente	
	GPS Service	
	└ Registra ubicación cada 30 segundos	
	Sync Worker (WorkManager)	
	└ Verifica conectividad cada 2 minutos	
	└ Envía datos pendientes al cloud	
	└ Retry automático si falla	

Flujo de Funcionamiento de la App

1. Inicio de Sesión

Usuario abre app



¿Token guardado? → SÍ → Auto-login → Dashboard



LoginScreen



Ingresa credenciales (DNI + contraseña)



API POST /api/v1/auth/login



Backend valida y retorna:

- Token JWT
- Datos del chofer
- ID de sesión



App guarda token en DataStore



Navega a Dashboard

2. Inicio de Monitoreo

DashboardScreen



Chofer presiona "Comenzar Viaje"



App solicita permisos:

- Cámara
- Ubicación (GPS)
- Audio (notificaciones)



Crea registro en Room Database:

- id_sesion
- id_chofer
- timestamp_inicio
- estado: "activo"



Inicia Foreground Service (AlertService)



Navega a MonitoringScreen

3. Monitoreo en Tiempo Real (Loop Principal)

LOOP CONTINUO (30 veces por segundo):

1. CameraX captura frame de cámara frontal



2. MediaPipe procesa frame:

- Detecta rostro
- Extrae 478 landmarks
- Calcula métricas:
 - * EAR (ojos)
 - * MAR (boca)
 - * Head pose (cabeza)



3. Algoritmo de Somnolencia analiza:

- SI $EAR < 0.2$ por >2 segundos:
 - Microsueño detectado
 - Incrementa contador
 - Trigger AlertService (HIGH)

SI MAR > 0.6 por >3 segundos:

- Bostezo detectado
- Incrementa contador
- Trigger AlertService (MEDIUM)

SI Head angle > 25° por >2 segundos:

- Cabeceo detectado
- Incrementa contador
- Trigger AlertService (MEDIUM)

↓

4. Guarda evento en Room Database:

```
INSERT INTO alertas (  
    timestamp,  
    tipo,  
    duracion,  
    latitud,  
    longitud,  
    sincronizado: false  
)
```

↓

5. Actualiza UI con métricas en tiempo real:

- Parpadeos/minuto
- Total microsueños
- Total bostezos
- Total cabeceos

↓

REPETIR cada 33ms (30 FPS)

4. Sistema de Alertas

Evento de somnolencia detectado

↓

AlertService recibe nivel de alerta:

- LOW: Parpadeo lento
- MEDIUM: Bostezo o cabeceo
- HIGH: Microsueño crítico

↓

SEGÚN NIVEL:

LOW:

- Notificación visual en pantalla

MEDIUM:

- Sonido de alerta moderado (500ms)
- Vibración corta (200ms)
- Notificación visual

HIGH:

- Sonido de alarma intenso (2 segundos)
- Vibración patrón largo (1 segundo)
- Notificación persistente
- Flash de pantalla roja

↓
Evento guardado en base de datos local

5. GPS Tracking

GPS Service activo en background

↓
CADA 30 SEGUNDOS:

1. FusedLocationProvider obtiene ubicación:

- Latitud
- Longitud
- Velocidad (km/h)
- Precisión (metros)

↓
2. Guarda en Room Database:

```
INSERT INTO ubicaciones (  
    timestamp,  
    latitud,  
    longitud,  
    velocidad,  
    sincronizado: false  
)
```

↓
3. Asocia ubicación a última alerta si existe

↓
REPETIR indefinidamente mientras sesión activa

6. Sincronización con Cloud

SyncWorker ejecuta cada 2 minutos

↓
1. Verifica conectividad:

```
ConnectivityManager.isNetworkAvailable()
```

↓
¿Hay internet? → NO → Cancela, reintenta en 2 min
↓ Sí

2. Consulta datos pendientes de sync:

```
SELECT * FROM alertas WHERE sincronizado = false  
SELECT * FROM ubicaciones WHERE sincronizado = false
```

↓
3. Agrupa datos en JSON:

```
{  
    "id_sesion": 123,  
    "alertas": [  
        {  
            "timestamp": "2025-11-15T14:30:00Z",  
            "tipo": "microsueño",  
            "duracion": 3.2,  
            "gps": {"lat": -12.04, "lon": -77.04}  
        },  
    ],  
}
```

```

    // ... más alertas
  ],
  "ubicaciones": [
    {"timestamp": "...", "lat": ..., "lon": ...},
    // ... más ubicaciones
  ]
}

```

↓

4. Envía a backend:

```

POST /api/v1/sync-data
Headers: Authorization: Bearer <token>
Body: JSON comprimido

```

↓

5. Backend responde:

```

HTTP 200 OK
{"status": "ok", "eventos_guardados": 15}

```

↓

6. Marca datos como sincronizados:

```

UPDATE alertas SET sincronizado = true WHERE id IN (...)
UPDATE ubicaciones SET sincronizado = true WHERE id IN (...)

```

↓

SI FALLA (sin internet o error servidor):

- Registra error
- Datos permanecen como "no sincronizados"
- WorkManager reintenta automáticamente
- Backoff exponencial: 2 min → 5 min → 10 min

7. Finalización de Viaje

Chofer presiona "Finalizar Viaje"

↓

1. Detiene todos los servicios:

- CameraX stop
- MediaPipe release
- AlertService stop
- GPS Service stop

↓

2. Actualiza sesión en Room:

```

UPDATE sesiones SET
  timestamp_fin = NOW(),
  estado = "finalizado",
  total_alertas = count(alertas)

```

↓

3. Trigger sincronización final inmediata:

```

SyncWorker.enqueueOneTime()

```

↓

4. Genera resumen local:

- Total tiempo de viaje
- Total microsueños
- Total bostezos
- Total cabeceos
- Puntos críticos GPS

- ↓
5. Muestra resumen en UI
- ↓
6. Navega a Dashboard

Base de Datos Local (Room/SQLite)

Entidades Principales

Tabla: sesiones

— id (PK)
— id_chofer
— timestamp_inicio
— timestamp_fin
— estado (activo/finalizado)
— total_alertas
— sincronizado

Tabla: alertas

— id (PK)
— id_sesion (FK)
— timestamp
— tipo (microsueño/bostezo/cabeceo)
— duracion_segundos
— latitud
— longitud
— velocidad_kmh
— sincronizado

Tabla: ubicaciones

— id (PK)
— id_sesion (FK)
— timestamp
— latitud
— longitud
— velocidad_kmh
— precision_metros
— sincronizado

Tabla: metricas_tiempo_real

— id (PK)
— id_sesion (FK)
— timestamp
— parpadeos_minuto
— ear_promedio
— mar_promedio
— angulo_cabeza

Pantallas de la Aplicación

1. SplashScreen

- Logo de la app
- Verifica si hay token guardado
- Auto-login si token válido
- Duración: 2 segundos

2. LoginScreen

- Campo: DNI/Código de chofer
- Campo: Contraseña
- Botón: "Iniciar Sesión"
- Checkbox: "Recordar sesión"
- Indicador de carga durante autenticación
- Mensajes de error si credenciales inválidas

3. DashboardScreen

- Saludo: "Hola, [Nombre Chofer]"
- Estadísticas del día:
 - Viajes completados hoy
 - Total de alertas
 - Tiempo total conducido
- Botón principal: "Comenzar Viaje" (grande, verde)
- Menú inferior:
 - Inicio
 - Historial
 - Configuración

4. MonitoringScreen (Principal)

Vista superior:

- Preview de cámara con landmarks dibujados
- Indicador de estado: "Monitoreando" (verde pulsante)

Vista central:

- Métricas en tiempo real:
 - 🕒 Tiempo de viaje: 01:23:45
 - 👁 Parpadeos/min: 18
 - 😴 Microsueños: 2
 - 🤤 Bostezos: 4
 - 📊 Inclinaciones: 1

Vista inferior:

- 📍 GPS: Activo (icono verde)
- 🌐 Sync: Última hace 1 min
- Botón: "Finalizar Viaje" (rojo)

Alertas emergentes:

- Banner rojo si microsueño crítico

- Banner amarillo si bostezo/cabeceo

5. HistoryScreen

- Lista de viajes pasados:
 - Fecha
 - Duración
 - Total alertas
 - Estado de sync
- Al hacer clic: Detalle del viaje
 - Timeline de alertas
 - Mapa de ruta GPS
 - Gráficos de métricas

6. SettingsScreen

- Sensibilidad de detección (Low/Medium/High)
- Volumen de alertas
- Intervalo de sincronización (1/2/5 minutos)
- Cerrar sesión
- Versión de la app



Configuración Técnica

Versiones Mínimas

Kotlin:

Gradle:

Android Gradle Plugin:

Compile SDK: 34 (Android 14)

Min SDK: 24 (Android 7.0)

Target SDK: 34

JVM Target:

Dependencias Críticas

Jetpack Compose:

CameraX:

MediaPipe Tasks Vision:

Room Database:

Retrofit:

Hilt:

WorkManager:

Google Play Services Location:



Integración con Backend FastAPI

Endpoints Utilizados

POST /api/v1/auth/login

└ Body: {"dni": "12345678", "password": "xxx"}
└ Response: {"token": "JWT...", "user": {...}}

POST /api/v1/sessions/start

└ Headers: Authorization: Bearer <token>
└ Body: {"chofer_id": 123}
└ Response: {"session_id": 456, "timestamp": "..."}

POST /api/v1/sync-data

└ Headers: Authorization: Bearer <token>
└ Body: {"alertas": [...], "ubicaciones": [...]}
└ Response: {"status": "ok", "eventos_guardados": 15}

POST /api/v1/sessions/end

└ Headers: Authorization: Bearer <token>
└ Body: {"session_id": 456}
└ Response: {"status": "ok", "resumen": {...}}

GET /api/v1/history/{chofer_id}

└ Headers: Authorization: Bearer <token>
└ Response: {"viajes": [...]}

Manejo de Tokens JWT

Al recibir token del login:



Guardar en DataStore encriptado



Incluir en cada request:

Header: Authorization: Bearer <token>



Si API retorna HTTP 401 Unauthorized:



Token expirado



Redirect a LoginScreen



Borrar token guardado

Optimización de Rendimiento

Procesamiento IA

Usar GPU del dispositivo:

- MediaPipe automáticamente usa GPU Android
- No requiere configuración adicional

Reducir resolución frames:

- Captura: 640x480 (suficiente para detección)
- No usar Full HD innecesario

Procesamiento cada N frames:

- Procesar 30 FPS (cada frame)
- Si dispositivo lento: Procesar 15 FPS (cada 2 frames)

Base de Datos

Limpieza automática de datos antiguos:

- Mantener solo últimos 30 días
- WorkManager ejecuta limpieza semanal

Índices en columnas críticas:

- timestamp
- id_sesion
- sincronizado

Manejo de Errores

Sin Conexión a Internet

App funciona 100% offline:

- Detección IA:  Funciona
- Alertas:  Funcionan
- GPS:  Funciona
- Guardar datos:  Funciona
- Sincronización:  En espera

Cuando recupera internet:

- SyncWorker envía todos los datos acumulados
- Usuario no necesita hacer nada

Cámara No Disponible

Si cámara falla:



Mostrar error: "Cámara no disponible"



Reintentar automáticamente 3 veces



Si sigue fallando:



Bloquear inicio de viaje



Sugerir reiniciar app o dispositivo

GPS No Disponible

Si GPS desactivado:



Mostrar diálogo: "Activa GPS para continuar"



Botón: "Ir a Configuración"



Usuario activa GPS



App detecta y continúa normal

Backend Caído

Si sincronización falla:



WorkManager reintentará con backoff:

- Intento 1: 2 minutos
- Intento 2: 5 minutos
- Intento 3: 10 minutos



Máximo 3 reintentos



Si sigue fallando:



Datos quedan guardados localmente



Próximo SyncWorker volverá a intentar



Conclusión

Esta aplicación Android representa la **capa crítica de seguridad** del sistema:

- ✓ **Garantiza protección al conductor** incluso sin internet
- ✓ **Alertas en tiempo real** con latencia <100ms
- ✓ **Sincronización inteligente** cuando hay conectividad
- ✓ **Arquitectura escalable y mantenible**
- ✓ **Tecnologías modernas y oficiales** de Google

Documentación Técnica - Proyecto Sistema de Detección de Somnolencia

Versión: 1.0

Tecnología: Kotlin + Jetpack Compose + MediaPipe

Plataforma: Android 7.0+ (API 24+)